

Chapter 3

Automating Lexical Analysis

Lexical analysis, part 2: from a regular-expression specification to a working, efficient lexer, mechanically

Compiler Construction Module

Based on Lecture 3, expanded

How to read this chapter

Chapter 2 ended with a promise: write the token rules as regular expressions and a tool will produce the lexer. This chapter keeps the promise. By the end you should be able to take any regular expression, build its NFA by hand, convert that NFA to a DFA, minimise the DFA, and explain how a table-driven lexer uses the result to cut a character stream into tokens with longest match. Every construction comes with a proof that it does what it claims, because in this subject “it usually works” is not good enough: a lexer that accepts one wrong string will eventually meet that string.

Three concept checks from the lecture appear as “Pause and check” boxes. Answer them before reading the solution; the solutions are where the understanding is.

1 The Pipeline

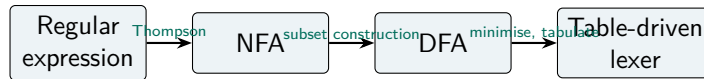
1.1 Motivation: what a lexer generator does

A lexer generator such as `flex` or `JFlex` takes a **lexical specification**, a list of regular expressions each paired with an action (“return the token `IF`”, “return `ID` with this text”), and produces code that breaks the input into tokens. Chapter 2 explained how to *write* such a specification. Today’s question is how the generator *converts* it into a program.

The key observation is that the whole specification is one regular expression,

$$R_1 \mid R_2 \mid \cdots \mid R_n,$$

where R_i describes token type i . A lexer generator converts this expression into a **deterministic finite automaton** (DFA) in two steps, and then runs the DFA from a table:



The regular expression is first converted into a **nondeterministic finite automaton** (NFA), because that conversion is easy and local; the NFA is then converted into a DFA, because a DFA can be executed by a tight loop over a table. The rest of the chapter works through each arrow: what the machines are, how each conversion goes, why it is correct, and what it costs.

Key insight. Each arrow is a small, mechanical algorithm with a correctness proof. Together they turn a declarative description into an executable one without a human writing a single `if`.

2 The Target Machine: DFAs

2.1 Motivation: a machine that never has to think

A DFA is the simplest possible recogniser. It reads one symbol at a time, keeps only a single number (its current state), and at every step has exactly one thing it can do. That is exactly the shape of a fast inner loop: no search, no backtracking, no memory beyond one integer. If we can compile our token rules into such a machine, lexing becomes a table lookup per character.

Concept: Deterministic finite automaton

Definition 2.1 (DFA). A DFA is a tuple $M = (Q, \Sigma, \delta, q_0, F)$ where

- Σ is a finite **alphabet** of input symbols;
- Q is a finite set of **states**;
- $q_0 \in Q$ is the **initial state**, where reading begins;
- $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**: on reading symbol x in state q the machine moves to $q' = \delta(q, x)$;
- $F \subseteq Q$ is the set of **accept states**. The machine accepts an input iff it ends the input in a state of F .

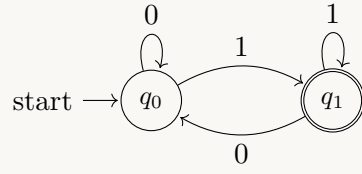
Diagrams draw states as circles, accept states as double circles, an arrow from nowhere into q_0 , and a labelled arrow $q \xrightarrow{x} q'$ for each transition.

Concept: The error state makes δ total

A diagram usually omits transitions for “illegal” symbols. Formally we add a non-accepting **error state** $\emptyset$ with $\delta(\emptyset, x) = \emptyset$ for every x , and send every otherwise-undefined transition to it. Then δ is defined on every (state, symbol) pair, so the reading loop never has to ask “is there a transition?”, and once the machine falls into $\emptyset$ it stays there and rejects.

Worked example: Binary strings ending in 1

The regular expression $(0 \mid 1)^*1$ describes binary strings whose last symbol is 1, i.e. odd numbers. A two-state DFA recognises it:



δ	0	1
q_0	q_0	q_1
q_1	q_0	q_1

$Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$. The table on the right is the whole implementation. Trace 1011: $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1 \in F$: accept. Trace 110: $q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_0 \notin F$: reject.

Mechanism: The reading loop

Algorithm 1: Run a DFA on $input[1..n]$

```

1  $q \leftarrow q_0$ ;
2  $i \leftarrow 1$ ;
3 while  $i \leq n$  do
4    $q \leftarrow \delta(q, input[i])$ ;
5    $i \leftarrow i + 1$ ;
6 if  $q \in F$  then return accept;
7 else return fail;

```

One table lookup per input character; time $O(n)$, memory one integer. This is why we want a DFA at the end of the pipeline.

2.2 What the loop computes, precisely

To prove anything about automata we need a name for “the state after reading a whole string.”

Definition 2.2 (Extended transition function; language of a DFA). Define $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ by $\hat{\delta}(q, \varepsilon) = q$ and $\hat{\delta}(q, wx) = \delta(\hat{\delta}(q, w), x)$ for a string w and symbol x . The **language** of M is $L(M) = \{w \in \Sigma^* : \hat{\delta}(q_0, w) \in F\}$.

The reading loop maintains the invariant “ $q = \hat{\delta}(q_0, input[1..i-1])$ ” (true initially with $i = 1$ and preserved by each iteration, by the recursive definition), so it returns *accept* exactly when $input \in L(M)$.

Proposition 2.3 (The two-state DFA is correct). *For the DFA above, $\hat{\delta}(q_0, w) = q_1$ if and only if w is nonempty and ends in 1. Hence $L(M) = L((0 \mid 1)^*1)$.*

Proof. Induction on $|w|$. For $w = \varepsilon$: $\hat{\delta}(q_0, \varepsilon) = q_0 \neq q_1$, and ε does not end in 1. For $w = ux$: by the table, $\delta(q, 1) = q_1$ and $\delta(q, 0) = q_0$ whatever q is, so $\hat{\delta}(q_0, ux) = q_1$ iff $x = 1$ iff ux ends in 1. $\square$

Key insight. The proof is short because the DFA “forgets” everything except the last symbol, which is all the language cares about. Good DFAs remember exactly what the language needs and nothing more; minimisation (Section 6)

2.3 The question

We can run a DFA from its table. But nobody wants to design a table by hand for $[a-zA-Z][a-zA-Z0-9]^*$, let alone for the union of forty token rules. The question is therefore: *how do we obtain the table δ from a regular expression?* The direct route is awkward, because regular expressions compose by union, concatenation and star, and DFAs do not compose easily under those operations. A machine that does compose easily is the subject of the next section.

3 The Bridge Machine: NFAs

3.1 Motivation: allow guessing, so that the pieces snap together

Suppose we have machines for R_1 and R_2 and want one for $R_1 \mid R_2$. The natural idea, “run both,” is not something a DFA can express with a single state. If instead we allow a machine to be in *several* states at once, or to *guess* which branch to take, gluing machines together becomes trivial. That is a nondeterministic automaton. It is not something we can execute directly with one integer, but it is the perfect intermediate representation: easy to build from a regex, and convertible to a DFA afterwards.

Concept: Nondeterministic finite automaton

Definition 3.1 (NFA). An NFA is a tuple $N = (Q, \Sigma, \Delta, q_0, F)$ like a DFA except that the transition relation $\Delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$ may contain, for a given state and symbol, *zero, one or several* target states, and may contain ε -**transitions** $q \xrightarrow{\varepsilon} q'$ that change state without reading anything. We write $q \xrightarrow{x} q'$ for $(q, x, q') \in \Delta$.

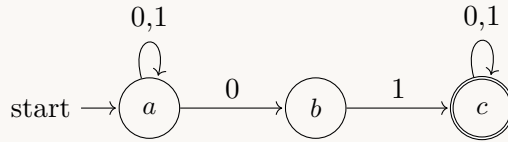
A string w is **accepted** if there exists *some* path from q_0 to a state in F whose non- ε labels, read in order, spell w . $L(N)$ is the set of accepted strings.

Two things deserve emphasis. First, acceptance is *existential*: one successful path suffices, however many dead paths there are. Second, every DFA is an NFA (with exactly one transition per pair and no ε), so NFAs are at least as expressive; Section 5 shows they are not more expressive.

Definition 3.2 (ε -closure). For a state q , $\text{closure}(q)$ is the set of states reachable from q by zero or more ε -transitions (so $q \in \text{closure}(q)$). For a set S , $\text{closure}(S) = \bigcup_{q \in S} \text{closure}(q)$.

Worked example: Pause and check: read a regular expression off an NFA

What single regular expression describes exactly the strings accepted by this NFA?



Solution. The machine may loop at a on anything, then at some moment guess “here comes 01”, move $a \xrightarrow{0} b \xrightarrow{1} c$, and loop at c on anything. So $L = (0 \mid 1)^* 01 (0 \mid 1)^*$: strings containing 01 as a substring. Note how nondeterminism expresses “somewhere” for free: at every 0 the machine can stay at a or move to b , and one of those guesses is right exactly when a 01 occurs. A DFA for the same language needs three states and a little care (it must remember “last symbol was 0”); the NFA needs no care at all.

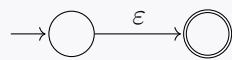
4 Regular Expression to NFA: Thompson’s Construction

4.1 Motivation: build by induction on the syntax

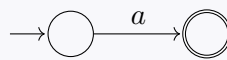
A regular expression is built from five constructs: ε , a symbol a , concatenation R_1R_2 , alternation $R_1 \mid R_2$, and star R^* (everything else in Chapter 2 was sugar for these). If we know how to build an NFA for each subexpression, we can build one for the whole expression by combining the sub-NFAs with a few ε -edges. To make the combining rules uniform, every NFA we build has **exactly one accept state** and no transitions *out* of that state; we draw such a fragment as a start circle, a cloud, and a single accept circle.

Mechanism: Thompson’s five rules

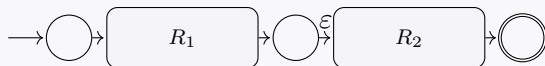
Each rule builds the NFA for an expression from the NFAs of its parts. New states are drawn filled; existing fragments are clouds.



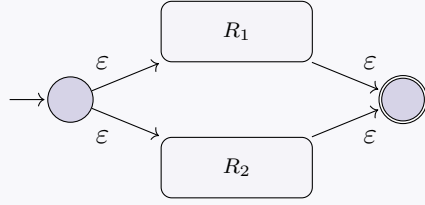
Rule ε



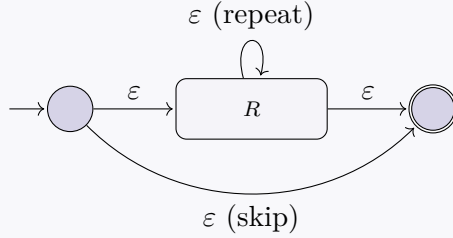
Rule a



Rule R_1R_2 : R_1 ’s accept stops being accepting; one ε -edge joins it to R_2 ’s start



Rule $R_1 \mid R_2$: new start and new accept



Rule R^* : new start and accept, skip edge, loop-back edge

In the star rule the loop-back edge runs from R 's accept state to R 's start state; in the drawing it is shown as a loop on the fragment.

Theorem 4.1 (Thompson's construction is correct). *For every regular expression R , the NFA $N(R)$ produced by the rules has a single accept state with no outgoing transitions, and $L(N(R)) = L(R)$.*

Proof. By structural induction on R . The single-accept, no-exit property is visible in each rule: base cases have it; concatenation keeps R_2 's accept (and R_1 's old accept now has one outgoing ε -edge but is no longer accepting); union and star introduce a fresh accept state with no exits. Now the language.

Rule ε , rule a . The only paths from start to accept spell ε , respectively a .

Concatenation. Any path from the new start to the new accept must pass through R_1 's old accept state, because that state is the only exit from the R_1 fragment into the R_2 fragment (its single ε -edge), and R_2 's fragment has no edge back. So a path spells w_1w_2 with w_1 spelled by a path through $N(R_1)$ from its start to its accept, and w_2 likewise in $N(R_2)$. By induction $w_1 \in L(R_1)$, $w_2 \in L(R_2)$, so $w \in L(R_1R_2)$; conversely any $w_1 \in L(R_1)$, $w_2 \in L(R_2)$ give accepting paths that join through the ε -edge.

Union. The new start has exactly two edges, both ε , into the two fragments, and the fragments have no edges between them; the new accept is reachable only from the two old accepts. So an accepting path enters exactly one fragment and traverses it from its start to its accept, spelling a string of $L(R_1)$ or of $L(R_2)$ by induction, and every such string is accepted.

Star. An accepting path either takes the skip edge (spelling ε), or enters R 's fragment, traverses it to its accept, and then either exits to the new accept or takes the loop-back edge to R 's start and traverses again. Since the fragment's accept is its only exit, the path decomposes as $w_1w_2 \cdots w_k$ ($k \geq 0$) with each $w_i \in L(R)$ by induction, hence in $L(R^*)$; and conversely every concatenation

of k strings of $L(R)$ has such a path. □

Proposition 4.2 (State count). *Let $\#leaves(R)$ be the number of occurrences of symbols and of ε in R , and $\#ops(R)$ the number of union and star operators. Then $N(R)$ has exactly $2(\#leaves + \#ops)$ states. Concatenation adds no states.*

Proof. Each base rule creates 2 states; union and star each create exactly 2 new states (a start and an accept) and never delete any; concatenation creates none, adding only one ε -edge. Sum over the syntax tree. □

Key insight. The single-accept-no-exit invariant is what makes the induction work: it guarantees that a path can leave a fragment only through the one door we control with an ε -edge.

4.2 Worked example: $(0 \mid 1)^*1$

Worked example: Step 0: parse the structure first

Before applying any rule, break the expression down the way the grammar would, because the rules apply inside-out:

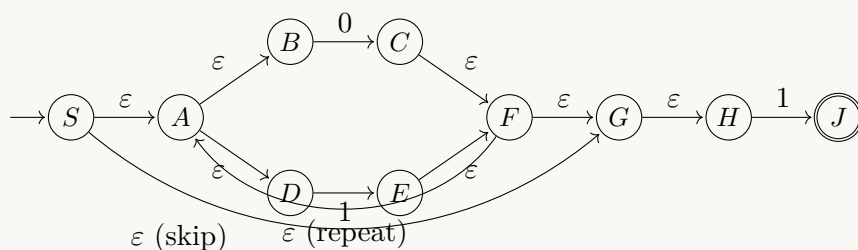
$$\begin{aligned} (0 \mid 1)^*1 &= E_1 \cdot E_2 && \text{concatenation, outermost} \\ E_1 &= E_0^* && \text{star} \\ E_0 &= 0 \mid 1 && \text{alternation} \\ E_2 &= 1 && \text{a single symbol} \end{aligned}$$

Build order, inside-out: symbol 0, symbol 1, alternation, star, symbol 1, concatenation.

Worked example: Steps 1–6: apply the rules; count as you go

Rule applied	States added	Running total
Symbol 0	+2 (B, C)	2
Symbol 1 (inner)	+2 (D, E)	4
Alternation $0 \mid 1$	+2 (A, F)	6
Star $(0 \mid 1)^*$	+2 (S, G)	8
Symbol 1 (outer)	+2 (H, J)	10
Concatenation	+0 (one ε -edge)	10

Proposition 4.2 agrees: 3 leaves and 2 operators give $2 \times 5 = 10$. The resulting NFA, with S the start and J the only accept state:



Read it as the rules dictate: $A \rightarrow F$ with the two branches is the alternation; S, G , the skip edge $S \rightarrow G$ and the loop-back $F \rightarrow A$ are the star; $G \rightarrow H$ is the concatenation's ε -edge; $H \xrightarrow{1} J$ is the final symbol. Trace 01: $S \xrightarrow{\varepsilon} A \xrightarrow{\varepsilon} B \xrightarrow{0} C \xrightarrow{\varepsilon} F \xrightarrow{\varepsilon} A \xrightarrow{\varepsilon} D \xrightarrow{1} E \xrightarrow{\varepsilon} F \xrightarrow{\varepsilon} G \xrightarrow{\varepsilon} H$: stuck, no more input, $H \notin F$. Another path: $S \rightarrow A \rightarrow B \xrightarrow{0} C \rightarrow F \rightarrow G \rightarrow H \xrightarrow{1} J$: accept. Existence of the second path is what counts.

Worked example: Pause and check 2: counting states in concatenation versus union

Suppose R_1 and R_2 each compile to a 2-state NFA (one start, one accept), like the base case for a single symbol. How many states do $R_1 R_2$ and $R_1 \mid R_2$ have?

Solution. Concatenation: $2+2=4$; the rule adds one ε -edge and no states (R_1 's accept simply stops being marked accepting). Union: $2+2+2=6$; a new start branching into both fragments and a new accept merging them back. Thompson's construction never merges two existing states into one; it only adds edges between fragments or fresh states around them. (A variant found in some texts identifies R_1 's accept with R_2 's start in concatenation, saving a state; the lecture's rules do not, which is why our count for $(0 \mid 1)^*1$ is 10.)

Warning / Gotcha: Common construction mistakes

- Adding a second accept state. Every fragment must have exactly one; otherwise the union and star rules do not know where to attach their edges.
- Letting an edge leave the accept state of a fragment before combining. The star rule adds the loop-back edge *from* the accept state; that is the only time an accept state gains an exit, and immediately afterwards it stops being the accept state.
- Forgetting the skip edge in the star rule: then R^* would not accept ε .
- Applying rules outside-in. Parse first (Step 0); the concatenation of $E_1 E_2$ cannot be built before E_1 and E_2 exist.

5 NFA to DFA: Subset Construction

5.1 Motivation: simulate every guess at once

An NFA accepts if *some* path works. Rather than search paths, keep track of *all* states the NFA could be in after each symbol. That set is a single, well-defined object determined by the input read so far, and the update from one set to the next is deterministic. So the sets of NFA states can serve as the states of a DFA. This is subset construction. Its cost is that there are potentially $2^{|Q|}$ subsets, which is why we build only those that are actually reachable.

Mechanism: Subset construction

Given an NFA $N = (Q, \Sigma, \Delta, q_0, F)$, define the DFA D whose states are subsets of Q :

start state: $Q_0 = \text{closure}(\{q_0\})$,

transition: $\delta_D(P, x) = \text{closure}(\{q' : q \in P, q \xrightarrow{x} q'\})$,

accept states: any P with $P \cap F \neq \emptyset$.

Compute Q_0 ; then repeatedly take an already-discovered set P and a symbol x , compute $\delta_D(P, x)$, and add it to the worklist if it is new. Stop when every discovered set has a transition for every symbol. The empty set, if it appears, is the error state.

Theorem 5.1 (Subset construction is correct). *For every string w , $\hat{\delta}_D(Q_0, w)$ is exactly the set of NFA states reachable from q_0 by a path spelling w . Consequently $L(D) = L(N)$.*

Proof. Induction on $|w|$. For $w = \varepsilon$, the states reachable while reading nothing are those reachable by ε -edges alone, i.e. $\text{closure}(\{q_0\}) = Q_0$. For $w = ux$, let $P = \hat{\delta}_D(Q_0, u)$, which by induction is the set of states reachable on u . A state is reachable on ux iff it is reachable from some $q \in P$ by first one x -edge (this consumes the last symbol; any ε -edges before it are already accounted for in P , since P is closed) and then zero or more ε -edges; that set is precisely $\text{closure}(\{q' : q \in P, q \xrightarrow{x} q'\}) = \delta_D(P, x)$. Finally $w \in L(N)$ iff some accept state is reachable on w iff $\hat{\delta}_D(Q_0, w) \cap F \neq \emptyset$ iff $w \in L(D)$. $\square$

Worked example: Subset construction on the NFA for $(0 \mid 1)^*1$

Start: $Q_0 = \text{closure}(\{S\})$. From S : ε to A and to G ; from A : to B , D ; from G : to H . So

$$Q_0 = \{S, A, B, D, G, H\}.$$

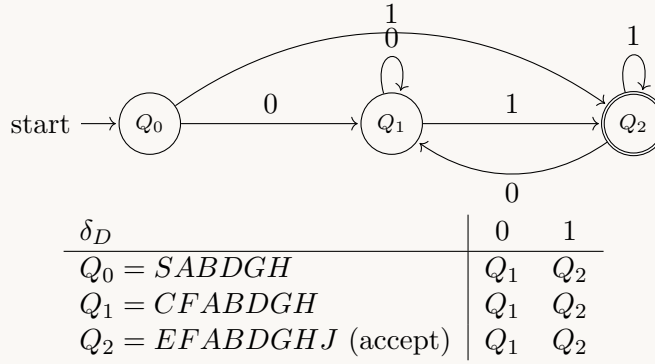
On 0: the only 0-edge from a state of Q_0 is $B \xrightarrow{0} C$. $\text{closure}(\{C\})$: $C \rightarrow F \rightarrow A \rightarrow B, D$ and $F \rightarrow G \rightarrow H$. So

$$Q_1 = \delta_D(Q_0, 0) = \{C, F, A, B, D, G, H\}.$$

On 1: 1-edges from Q_0 : $D \xrightarrow{1} E$ and $H \xrightarrow{1} J$. $\text{closure}(\{E, J\})$: $E \rightarrow F \rightarrow A, G \rightarrow B, D, H$; J has no ε -edges. So

$$Q_2 = \delta_D(Q_0, 1) = \{E, F, A, B, D, G, H, J\}, \quad \text{accepting since } J \in Q_2.$$

From Q_1 : its 0-edges are again only $B \rightarrow C$, giving Q_1 ; its 1-edges are $D \rightarrow E$ and $H \rightarrow J$, giving Q_2 . From Q_2 : the same two computations give Q_1 on 0 and Q_2 on 1. No new sets appear; stop.



Three DFA states out of $2^{10} = 1024$ possible subsets. Notice that Q_0 and Q_1 have identical rows: they behave the same on every input. Section 6 will merge them.

Technique: Doing subset construction without losing track

- Compute ε -closures with a worklist: write the seed states, then keep adding ε -targets of states already listed until nothing new appears.
- Sort or canonically order the members of each set so that you recognise a set you have already built. $\{C, F, A, \dots\}$ and $\{A, B, C, \dots\}$ are the same state.
- Process one row of the table at a time: for the current set, for each symbol, collect targets, then close.
- An accept state in the DFA is any set that *contains* an NFA accept state, however many other states it contains.

5.2 The cost: exponential blow-up

Worked example: Pause and check 1: why not go straight from regex to DFA?

The pipeline goes $\text{regex} \rightarrow \text{NFA} \rightarrow \text{DFA}$ even though subset construction could, in principle, be folded into the translation. What is the real cost of skipping the NFA stage?

Solution. Subset construction can, in the worst case, produce a DFA with exponentially more states than the NFA: each DFA state is a set of NFA states, and an n -state NFA has 2^n subsets. Isolating this step keeps the exponential bookkeeping out of the regex translation, which is linear and local (Proposition 4.2). The claims “there is no real cost” and “a DFA can never have more states than the regex has symbols” are both false; the next proposition gives a family where the blow-up actually happens.

Proposition 5.2 (Exponential blow-up is real). *Let $L_n = L((0 \mid 1)^* 0 (0 \mid 1)^{n-1})$, the binary strings whose n -th symbol from the end is 0. There is an NFA for L_n with $n + 1$ states, but every DFA for L_n has at least 2^n states.*

Proof. NFA. States $p_0, \dots, p_n$; loop at p_0 on 0, 1; $p_0 \xrightarrow{0} p_1$; $p_i \xrightarrow{0,1} p_{i+1}$ for

$1 \leq i < n$; accept p_n . The machine guesses when the last n symbols begin.

DFA lower bound. Let M be any DFA for L_n and consider the 2^n strings u of length n . Suppose two distinct ones, $u \neq u'$, led to the same state: $\hat{\delta}(q_0, u) = \hat{\delta}(q_0, u')$. They differ at some position i (counting from 1); say $u_i = 0$ and $u'_i = 1$. Append $v = 1^{i-1}$. Then in uv the n -th symbol from the end is $u_i = 0$, so $uv \in L_n$, while $u'v \notin L_n$. But from a common state the same suffix v leads to the same final state, so M accepts both or neither: contradiction. Hence the 2^n strings lead to 2^n distinct states. $\square$

In practice, token specifications rarely trigger the blow-up; lexer generators either build the DFA lazily or accept the cost, because the DFA is built once and then run on millions of characters.

Key insight. The DFA must remember the last n symbols in full, because any of them might turn out to be the decisive one; the NFA instead guesses which one is decisive and remembers only a counter. Determinism costs memory.

6 Cleaning Up: DFA Minimisation

6.1 Motivation: smaller tables, and a canonical answer

Subset construction produced three states for a language that needs two. Extra states cost table space and cache misses in the lexer, and, more importantly, they hide the structure: two DFAs for the same language may look nothing alike. Minimisation fixes both. By the Myhill–Nerode theorem, every regular language has a *unique* minimal DFA (up to renaming of states), and it is obtained from any DFA by merging states that “behave the same.”

Concept: Equivalent and distinguishable states

Definition 6.1. States q_1, q_2 of a DFA are **equivalent**, $q_1 \approx q_2$, if for every string s , $\hat{\delta}(q_1, s) \in F \iff \hat{\delta}(q_2, s) \in F$: starting from either, the machine accepts exactly the same remaining suffixes. Otherwise they are **distinguishable**, and a string s accepted from one but not the other is a **witness**.

Mechanism: Table-filling algorithm

Rule 1. If one of q_1, q_2 is accepting and the other is not, mark the pair distinguishable; the witness is ε .

Rule 2. If for some symbol x the pair $(\delta(q_1, x), \delta(q_2, x))$ is already marked, mark (q_1, q_2) ; if s witnessed the successors, xs witnesses (q_1, q_2) . Apply Rule 1 to all pairs, then sweep Rule 2 over all unmarked pairs repeatedly until a full sweep marks nothing. Every pair still unmarked is equivalent: merge them.

Theorem 6.2 (The table-filling algorithm is correct). *When the algorithm stops, a pair is marked if and only if it is distinguishable.*

Proof. *Marked $\Rightarrow$ distinguishable.* Induction on the order in which pairs are marked. Rule 1 marks pairs distinguished by ε . Rule 2 marks (q_1, q_2) only

when $(\delta(q_1, x), \delta(q_2, x))$ was marked earlier, hence (induction) distinguished by some s ; then xs distinguishes (q_1, q_2) , since $\hat{\delta}(q_j, xs) = \hat{\delta}(\delta(q_j, x), s)$.

Unmarked $\Rightarrow$ equivalent. Let U be the set of unmarked pairs at termination. U has two properties: (a) no pair in U has exactly one accepting member (Rule 1 would have marked it); (b) for every $(q_1, q_2) \in U$ and every symbol x , $(\delta(q_1, x), \delta(q_2, x)) \in U$ (otherwise the final sweep of Rule 2 would have marked (q_1, q_2)). Now take $(q_1, q_2) \in U$ and any string s ; by (b) applied symbol by symbol, $(\hat{\delta}(q_1, s), \hat{\delta}(q_2, s)) \in U$, and by (a) both or neither of these are accepting. So s does not distinguish them; as s was arbitrary, $q_1 \approx q_2$. $\square$

Proposition 6.3 (Merging is safe). *Let $\approx$ be the equivalence relation of unmarked pairs. The quotient DFA whose states are the $\approx$ -classes, with $\delta([q], x) = [\delta(q, x)]$, start $[q_0]$ and accept states $\{[q] : q \in F\}$, is well defined and accepts the same language. (It is in fact the unique minimal DFA, which is the Myhill–Nerode theorem; we do not prove minimality here.)*

Proof. Well defined: if $q \approx q'$ then $\delta(q, x) \approx \delta(q', x)$, because a witness s for the successors would give a witness xs for (q, q') ; so the transition does not depend on the representative. Acceptance is well defined by property (a) above. For any string w , induction on $|w|$ gives $\hat{\delta}_{\text{quot}}([q_0], w) = [\hat{\delta}(q_0, w)]$, and $[q]$ is accepting iff q is; so the languages coincide. $\square$

Worked example: Minimising the three-state DFA, step by step

States $Q_0 = SABDGH$, $Q_1 = CFABDGH$, $Q_2 = EFABDGHJ$ (accepting). Three pairs.

Steps 1–2, Rule 1.

Pair	Result	Why
(Q_0, Q_1)	unmarked	both non-accepting; Rule 1 says nothing
(Q_0, Q_2)	marked, witness ε	Q_2 accepting, Q_0 not
(Q_1, Q_2)	marked, witness ε	Q_2 accepting, Q_1 not

Steps 3–4, Rule 2 sweeps. Only (Q_0, Q_1) survives. On 0: $Q_0 \rightarrow Q_1$ and $Q_1 \rightarrow Q_1$, the same state, nothing to look up. On 1: both go to Q_2 , again the same state. Rule 2 cannot fire. A second sweep finds the DFA unchanged, so the result is identical: stop.

Step 5, read off the table.

	Q_0	Q_1
Q_1	unmarked	
Q_2	ε (Rule 1)	ε (Rule 1)

Q_0 and Q_1 are each distinguishable from Q_2 but not from each other: merge them.

Worked example: How merging actually works

Fold Q_0 and Q_1 into one state M . This is safe only because they agree on every transition, which is exactly what the sweep proved:

	0	1
Q_0	Q_1	Q_2
Q_1	Q_1	Q_2

Either row serves for M : M on 0 $\rightarrow Q_1$, M on 1 $\rightarrow Q_2$. Then *relabel every mention of a merged state*, from anywhere in the table:

Transition	Originally	Rewritten
M on 0	$\rightarrow Q_1$	$\rightarrow M$ (self-loop; Q_1 was folded into M)
M on 1	$\rightarrow Q_2$	$\rightarrow Q_2$ (unaffected)
Q_2 on 0	$\rightarrow Q_1$	$\rightarrow M$ (relabelled)
Q_2 on 1	$\rightarrow Q_2$	$\rightarrow Q_2$ (unaffected)

Start: Q_0 was the start and is now part of M , so M is the start. Accept: Q_2 stays accepting; M contains no accepting state, so it is not. Rename $M \rightarrow q_0$, $Q_2 \rightarrow q_1$: exactly the two-state DFA we began the chapter with. The pipeline went full circle, and the minimal DFA is the same however we arrived at it, which is the content of Myhill–Nerode.

Warning / Gotcha: Minimisation pitfalls

- Do not merge states just because their rows *look* similar; merge only unmarked pairs. Two states with rows (Q_1, Q_2) and (Q_1, Q_3) are not equivalent if $Q_2 \not\approx Q_3$.
- Include the error state in the table, or make sure every state has a transition on every symbol; a missing transition is a transition to $\emptyset$, and $\emptyset$ is distinguishable from every accepting state.
- Remove unreachable states first; the algorithm says nothing about them and they would appear as spurious equivalence classes.
- After merging, relabel *all* transitions pointing at the merged states, not just the merged state's own row.

7 From One Token to Many

7.1 Motivation: which rule matched?

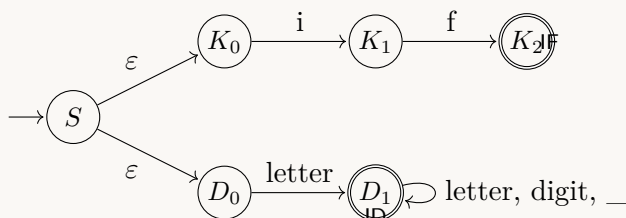
A real specification is $R_1 \mid R_2 \mid \dots \mid R_n$, and knowing that the input “matched” is not enough: the lexer must know *which* R_i matched, because each carries a different action. Two changes to the pipeline achieve this.

Mechanism: Combined NFA with tagged accept states

Build one big alternation NFA with a new start state and an ε -edge into each $N(R_i)$, but *do not* merge the accept states: each branch keeps its own accept state, tagged with its action. When subset construction lands in a DFA state containing accept states from several branches, the state is tagged with the action of the *highest-priority* branch, where priority is the order of the rules in the specification (earlier wins).

Worked example: Two branches: the keyword `if` and identifiers

Consider just the branches for the literal `if` and for $\{Identifier\} = letter(letter | digit | _)*$:



Trace the input `if` through both branches at once:

after <code>i</code>	$\{K_1, D_1\}$, and D_1 accepts (ID)
after <code>if</code>	$\{K_2, D_1\}$: <i>both</i> accept (K_2 tags IF, D_1 tags ID)

The DFA state after `if` contains two accepting NFA states with different actions; a lexer can return only one token. The tie-break is rule order: "`if`" was written before $\{Identifier\}$ in the spec, so IF wins.

Proposition 7.1 (Rule order only breaks ties in length). *The lexer first finds the longest prefix of the remaining input that is accepted by the combined DFA (Section 8); only then, if that prefix is accepted through several branches, is the branch chosen by priority. Hence rule order matters only when two rules match the same characters.*

Proof. Longest match is determined by the DFA's accepting states as sets, independently of tags; the tag of the final state is consulted once, after the length is fixed. Two rules with matches of different lengths never compete for the same DFA state at the same position. $\square$

Worked example: Why this does not contradict maximal munch

Input	Competing matches	Length check	Resolved by
<code>if</code> alone	IF: 2 chars; ID: 2 chars	tie	rule order: IF
<code>infinity</code>	IF: 2 chars; ID: 8 chars	no tie ($8 > 2$)	longest match: ID

Longest match wins whenever it can; rule order is the tie-breaker of last

resort. This is exactly why keyword rules are always written as exact literal strings and placed before the identifier rule.

8 Longest Match: The Scan-and-Backtrack Algorithm

8.1 Motivation: a DFA answers “is this whole string a token?”; we need “where does the first token end?”

The DFA reading loop decides membership of a complete string. A lexer faces a different question: given the remaining input, what is the *longest* prefix that forms a token, and of which type? The answer is not “stop at the first accepting state”: after **in** the DFA accepts (an identifier), but the token is **infinity**. Nor is it “read until stuck”: on **abc** followed by **d** with rules **abc** and **(abc)*d**, reading greedily may run into a dead end past the last valid token. The algorithm below reads ahead greedily and then backtracks to the last accepting position.

Mechanism: Scan and backtrack

Algorithm 2: Longest-match tokenisation from position i (Reps’ formulation)

```

1  $start \leftarrow i$ ;  $q \leftarrow q_0$ ; clear the stack;
2 Phase 1: read ahead until stuck;
3 while true do
4    $input[i] \leftarrow read()$ ;
5   if  $input[i] = EOF$  or  $\delta(q, input[i]) = \emptyset$  then break;
6   if  $q \in F$  then clear the stack;
7   push  $q$ ;
8    $q \leftarrow \delta(q, input[i])$ ;  $i \leftarrow i + 1$ ;
9 Phase 2: backtrack to the last accepting state;
10 while  $q \notin F$  do
11   if stack is empty then fail;
12    $q \leftarrow pop()$ ;
13   unread( $input[i]$ );  $i \leftarrow i - 1$ ;
14 return  $input[start..i - 1]$ , with the action of  $q$ ;
```

Theorem 8.1 (Scan-and-backtrack returns the longest match). *If some prefix of the remaining input is accepted by the DFA, the algorithm returns the longest such prefix; otherwise it fails. When it returns, i is one past the end of the token, so $input[start..i - 1]$ is the token’s text.*

Proof. Invariant of Phase 1. After k iterations, $q = \hat{\delta}(q_0, input[start..start + k - 1])$, and the stack holds, bottom to top, the states $q_j, q_{j+1}, \dots, q_{k-1}$ where

Key insight. The stack is cleared every time an accepting state is left. So at any moment it holds exactly the states visited since the most recent accept. Backtracking pops through them and lands on that accept: the longest match, by construction.

q_m denotes the state after m symbols and j is the largest index $\leq k-1$ with $q_j \in F$ (or $j = 0$ if none; and if q_{k-1} itself is accepting the stack holds just q_{k-1}). This holds initially and is preserved: each iteration pushes the current state, after first clearing the stack if that state is accepting, which resets j to the current index.

Phase 1 stops at the first k such that the next symbol is EOF or leads to the error state $\emptyset$. Since $\emptyset$ is absorbing and non-accepting, no prefix longer than k symbols can be accepted: every longer prefix passes through $\emptyset$. So the longest accepted prefix, if any, has length $\leq k$, and it is the largest $m \leq k$ with $q_m \in F$.

Phase 2 pops states in the order $q_{k-1}, q_{k-2}, \dots$ while unread one symbol each time, so after popping down to q_m the position is $i = \text{start} + m$. It stops at the first accepting state encountered, which by the invariant is q_j with j the largest accepting index $< k$ (or q_k itself if accepting, in which case the loop does not run). Either way it stops at the largest accepting $m \leq k$, the longest match. If the stack empties before an accepting state is found, no prefix of length $\leq k$ is accepted, hence none at all: fail is correct. $\square$

Worked example: A trace with real backtracking

Rules: $R_1 = \text{abc}$ and $R_2 = (\text{abc})^* \text{d}$. Input: `abcabx`.

read	state after	accepting?	stack after the step	note
a	q_a	no	$[q_0]$	
b	q_{ab}	no	$[q_0, q_a]$	
c	q_{abc}	yes (R1; R2 still needs d)	$[q_0, q_a, q_{ab}]$	
a	q_{abca}	no	$[q_{abc}]$	stack cleared because $q_{abc} \in F$, then q_{abc} pushed
b	q_{abcab}	no	$[q_{abc}, q_{abca}]$	
x	$\emptyset$			stuck: break before pushing

Phase 2: $q_{abcab} \notin F$: pop q_{abca} , unread **b**; still not accepting: pop q_{abc} , unread **a**; $q_{abc} \in F$: stop. Return `abc` with action R1; the lexer resumes at `abx`. Reading three characters past the end of the token and giving them back is the price of not knowing, at `abc`, whether a `d` was coming.

8.2 Keeping it linear: the quadratic trap and Reps' fix

Worked example: The quadratic trap

Same rules, input $(\text{abc})^k$ with no `d` anywhere ($n = 3k$ characters). For the first token, Phase 1 reads all the way to the end hoping for a `d`, then backtracks $n - 3$ characters. For the second token, again to the end and back $n - 6$. In total k tokens, each costing $\Theta(n)$: $\Theta(n^2)$ work for n characters of input, all of it re-reading the same suffix and re-discovering that it leads nowhere.

Mechanism: Reps' fix: memoise hopeless states

If during backtracking the algorithm passes through a non-accepting state q at position i , then starting *any* future read-ahead from (q, i) is pointless: from that configuration no accepting state was reached before getting stuck, and the DFA and the input have not changed. Record it in a table $hopeless[q, i]$ and consult it before advancing:

```
while (true) {
    if (hopeless[q, i]) break;           // new
    input[i] = read();
    if (input[i] == EOF || delta(q, input[i]) == EMPTY)
        break;
    ...                                 // as before
}
while (q not in F) {
    hopeless[q, i] = true;               // new
    ...                                 // as before
}
```

Proposition 8.2 (With memoisation, tokenisation is linear). *With the hopeless table, the total number of Phase 1 iterations over the whole input is at most $|Q| \cdot n$, where n is the input length, and the result is unchanged.*

Proof. Unchanged result. Breaking at a hopeless (q, i) only shortens a read-ahead that would have ended in backtracking through (q, i) anyway, and backtracking then proceeds from the same stack contents as before, landing on the same last accepting state.

Linearity. Each Phase 1 iteration pushes a configuration (q, i) (the state before the advance and the position). A pushed configuration is either eventually popped during backtracking, at which point it is marked hopeless and can never be pushed again (the check precedes every advance), or it is discarded by a stack clear, which happens only when the machine is *in* an accepting state at position i ; but after a clear the token returned ends at or beyond that position, and the lexer never returns to positions before the end of a returned token. So each (q, i) is pushed at most once over the whole run: at most $|Q| \cdot n$ iterations. $\square$

9 One More Option: Recognising Without a DFA

Precompiling to a DFA pays off when the specification is fixed, as in a compiler. For one-off matching of a user-supplied pattern, three alternatives exist.

In the real world. On $(abc)^k$ only the first token pays the full read-ahead; every later abc finds the states past its own end already marked hopeless and stops immediately. Reps, “Maximal-munch tokenization in linear time,” TOPLAS 1998.

Trade-offs: Three ways to run a regular expression

- **Backtracking over the NFA** (Java, Perl, Python `re`). Try one path; on failure, undo and try the next. Simple and supports extensions (backreferences) beyond regular languages, but *exponential* in the worst case: on $(a \mid a)^*b$ against a^n there are 2^n ways to consume the a 's, all tried before failing.
- **NFA simulation** (Thompson's original, RE2). Keep the *set* of current states and update it per symbol: exactly one step of subset construction per character, computing only the DFA states actually needed and caching them. Linear in the input, no backtracking, no exponential blow-up up front.
- **Derivatives** (Brzozowski). Differentiate the regular expression itself, as below.

Concept: Brzozowski derivatives

Definition 9.1. The **derivative** of a language L with respect to a symbol a is $\partial_a L = \{w : aw \in L\}$: what remains of L after consuming an a . On regular expressions it is computed syntactically, with $\nu(R) = \varepsilon$ if $\varepsilon \in L(R)$ and $\nu(R) = \emptyset$ otherwise (the *nullability* of R):

$$\begin{aligned} \partial_a a &= \varepsilon, & \partial_a b &= \emptyset \ (b \neq a), \\ \partial_a \varepsilon &= \emptyset, & \partial_a \emptyset &= \emptyset, \\ \partial_a (R_1 \mid R_2) &= \partial_a R_1 \mid \partial_a R_2, & \partial_a (R^*) &= (\partial_a R) R^*, \\ \partial_a (R_1 R_2) &= (\partial_a R_1) R_2 \mid \nu(R_1) \partial_a R_2. \end{aligned}$$

The concatenation rule says: either the a is consumed by R_1 , or R_1 matched the empty string and the a is consumed by R_2 . A regular expression R accepts $w = a_1 \cdots a_n$ iff $\partial_{a_n} \cdots \partial_{a_1} R$ is nullable.

Worked example: Derivatives build the minimal DFA directly

Let $R = (0 \mid 1)^*1$. Then

$$\begin{aligned} \partial_0 R &= (\partial_0(0 \mid 1)^*) 1 \mid \nu((0 \mid 1)^*) \partial_0 1 \\ &= ((\partial_0 0 \mid \partial_0 1)(0 \mid 1)^*) 1 \mid \varepsilon \emptyset \\ &= ((\varepsilon \mid \emptyset)(0 \mid 1)^*) 1 \equiv R, \\ \partial_1 R &= ((\partial_1 0 \mid \partial_1 1)(0 \mid 1)^*) 1 \mid \varepsilon \partial_1 1 \\ &= ((\emptyset \mid \varepsilon)(0 \mid 1)^*) 1 \mid \varepsilon \equiv R \mid \varepsilon. \end{aligned}$$

So consuming a 0 leaves us with R again, and consuming a 1 leaves $R \mid \varepsilon$. From $R \mid \varepsilon$: $\partial_0(R \mid \varepsilon) = \partial_0 R \mid \emptyset \equiv R$ and $\partial_1(R \mid \varepsilon) \equiv R \mid \varepsilon$. Two expressions, closed under derivatives; R is not nullable, $R \mid \varepsilon$ is. Taking the expressions as states,

$$R \xrightarrow{0} R, \quad R \xrightarrow{1} R \mid \varepsilon, \quad R \mid \varepsilon \xrightarrow{0} R, \quad R \mid \varepsilon \xrightarrow{1} R \mid \varepsilon,$$

with $F = \{R \mid \varepsilon\}$, which is the two-state DFA of Proposition 2.3, obtained with no NFA, no subset construction and no minimisation. The catch: recognising that two expressions are “the same” state requires simplification (we used $\varepsilon R \equiv R$, $\emptyset R \equiv \emptyset$, $R \mid \emptyset \equiv R$); with similarity up to those identities the construction is guaranteed to terminate.

10 Tricks, Techniques and Classic Problems

Technique: Exam-speed recipes

- **Regex $\rightarrow$ NFA:** parse first, build inside-out, two new states per symbol/ ε /union/star, none per concatenation. Check the count with Proposition 4.2.
- **NFA $\rightarrow$ DFA:** closure of the start; one row per discovered set; a set is accepting iff it contains an NFA accept state; stop when no new set appears.
- **Minimise:** Rule 1 on all pairs, then Rule 2 sweeps until stable; merge unmarked pairs; relabel all transitions.
- **Which DFA is right?** Two DFAs for the same language minimise to the same machine. Minimise both and compare.
- **Lower bound on DFA size:** exhibit k strings pairwise distinguishable by some suffix; then every DFA needs $\geq k$ states (the argument of Proposition 5.2).
- **Longest match by hand:** read until stuck, then walk back to the last accept; the stack is just “the positions since the last accept.”

Worked example: Classic problems

P1. How many states does Thompson’s NFA for $(a \mid b)^*abb$ have?

Leaves: a, b, a, b, b (5); operators: one union, one star (2). Total $2 \times 7 = 14$.

P2. Give the minimal DFA for strings over $\{0, 1\}$ with an even number of 1s.

Two states (parity), both toggled by 1, unchanged by 0; the even state is start and accept. It is minimal because ε distinguishes the two states.

P3. Why does the NFA for L_3 (third symbol from the end is 0) have 4 states while any DFA has ≥ 8 ?

Proposition 5.2 with $n = 3$: the 8 strings of length 3 are pairwise distinguishable.

P4. Subset construction gives a DFA state that is the empty set. What is it?

The error state $\emptyset$: no NFA path survives. It loops to itself on every symbol and is never accepting.

P5. In the table-filling algorithm, is a witness for (q_1, q_2) always of length $\leq |Q| - 2$?

Yes: pairs marked in the k -th Rule-2 sweep have witnesses of length k , and there are at most $\binom{|Q|}{2}$ pairs, but more sharply each sweep must mark

a new pair or stop, and a witness grows by one symbol per sweep; the classical bound is $|Q| - 2$.

P6. Rules "<", "<=", "<<", "<=" in this order; input a<=b. Which token after a?

Longest match: <= (3 characters) beats <<, <=, <. Rule order never enters: no tie in length.

P7. Input if with rules {Identifier} first and "if" second. Token?

Tie in length; earlier rule wins: ID("if"). The keyword is lost. This is why keyword rules go first.

P8. Why is clearing the stack on leaving an accepting state essential?

Without it, backtracking would stop at the *first* accepting state popped from the top, which is the most recent one anyway; but the stack would grow with the entire read-ahead, and with memoisation the clear is what bounds pushes per position. In the lecture's formulation the clear is what makes "pop until accepting" land on the longest match rather than requiring a separate search.

P9. $\partial_a(a^*b)$?

$(\partial_a a^*)b \mid \nu(a^*)\partial_a b = (\varepsilon a^*)b \mid \varepsilon \emptyset \equiv a^*b$. The state is unchanged by reading a , as it should be.

Warning / Gotcha: Gotchas

- An NFA accepts if *some* path accepts; a dead path proves nothing. Students often reject a string after following one unlucky path.
- The DFA state after subset construction is a *set*; do not write it as a single NFA state.
- "Minimal" means fewest states for the *same language*. You cannot minimise by dropping accept states or transitions you dislike.
- The scan-and-backtrack algorithm may read past the token; the characters must be *unread*, not lost. A lexer that discards them mis-tokenises the next token (compare Chapter 2's `read()` bug).
- Rule order is not a substitute for longest match; it applies only to equal-length matches.

Notes on the original slides

- **State count: 9 or 10?** Slide 7 tabulates 10 states for the NFA of $(0 \mid 1)^*1$ (concatenation adds none), and slide 8 draws all ten ($S, A, B, C, D, E, F, G, H, J$). The concept-check solution on slide 12 says the NFA "ends up with 9 states." The rules as stated on slide 5 give 10; the count 9 corresponds to the variant of concatenation that identifies R_1 's accept with R_2 's start (used in the Cornell notes the lecture refers to). Both are valid constructions; be consistent with whichever rule set you use, and say which.
- **Derivative rules (slide 33) are incomplete.** The slide lists $\partial_a a$, $\partial_a \varepsilon$, $\partial_a(R_1 + R_2)$ and $\partial_a(R^*)$ but omits the concatenation rule and the nullability

function ν , without which no expression containing a concatenation can be differentiated. The full rule set is given in the derivatives box above.

- **Slide numbering.** Slides carry page numbers from two different orderings (e.g. “22”, “23” on the concept-check slides between “8” and “9”); the concept checks were evidently inserted later. Nothing is missing; read them where they make sense (after Thompson’s construction and after subset construction).
- **Slide 3, “ as a table.”** The table is δ ; δ is used elsewhere for the NFA relation. Same object here.
- **Slide 29.** “exactly the $abc \mid (abc)^*d$ scenario” refers forward to slide 32; the trace in Section 8 shows it concretely.
- **Termination of the derivative construction** (slide 33) is not discussed on the slide; it requires simplifying expressions up to associativity, commutativity and idempotence of $\mid$ (Owens, Reppy and Turon 2009), as noted in the derivatives example.

Summary: Chapter 3 in seven lines

1. A lexer generator compiles $R_1 \mid \dots \mid R_n$ into a DFA and runs it from a table; each arrow of the pipeline is a mechanical algorithm.
2. A DFA is one integer of memory and one lookup per character; an NFA may guess, and accepts if some path accepts.
3. Thompson’s construction builds an NFA by induction on the regex: two states per leaf, union or star; none per concatenation; single accept state.
4. Subset construction runs all guesses at once: DFA states are sets of NFA states; correct by induction; exponential in the worst case, and that worst case is real.
5. Table filling marks distinguishable pairs (Rule 1: acceptance; Rule 2: successors); unmarked pairs are equivalent and merge into the unique minimal DFA.
6. A combined DFA tags accepting states with the highest-priority rule; longest match is found by reading ahead and backtracking to the last accept; rule order breaks only ties.
7. Backtracking can be quadratic; memoising hopeless (q, i) pairs makes it linear. Alternatives: NFA simulation, and derivatives, which build the minimal DFA directly.